

РАБОТЕН ЛИСТ: Упражнение №0

Дисциплина: Киберсигурност

Тема: Представяне на данни, XOR и Шифър на Цезар

Име на студент:

Факултетен номер:

Подготовка

1. Отворете вашата IDE (VS Code или др.) или текстов редактор.
2. Създайте нов файл, с посоченото име на задачата.
3. Използвайте приложените по-долу кодови скелети ("Code Skeletons"), като попълвате местата, означени с `// TODO`.

ЗАДАЧА 1: Data Representation (ASCII & Hex) - ascii_to_hex.c

Цел: Да се научим да виждаме данните "под капака" – като числови стойности (Decimal) и байтове (Hex).

Условие:

Напишете програма, която дефинира текстов низ (стринг) `message[] = "Cyber"`.

Програмата трябва да обходи низа и за всеки символ да отпечата:

1. Самия символ.
2. Десетичната му стойност (ASCII).
3. Шестнадесетична му стойност (HEX).

Скелет на кода:

C

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

```
void print_hex(const char *text) {
    printf("Symbol | Decimal | Hex\n");
    printf("-----|-----|----\n");
```

```

    for (int i = 0; i < strlen(text); i++) {
        // TODO: Използвайте printf с формати %c, %d и %02X
    }
}

int main() {
    char message[] = "Cyber";
    print_hex(message);
    return EXIT_SUCCESS;
}

```

ЗАДАЧА 2: Магията на XOR (The Perfect Cipher Logic) - xor_cipher.c

Цел: Да демонстрираме свойството на обратимост на XOR операцията:
 $(A \oplus K) \oplus K = A$.

Условие:

Реализирайте функция, която приема входен текст и един символ за ключ. Функцията трябва да приложи XOR операция върху всеки байт от текста с ключа.

1. Криптирайте текста "Secret" с ключ 'K'.
2. Принтирайте криптирания резултат (ще изглежда като "боклук").
3. Приложете същата функция върху криптирания резултат със същия ключ 'K'.
4. Проверете дали сте получили обратно "Secret".

Скелет на кода:

```

C
void xor_cipher(char *text, char key) {
    int len = strlen(text);
    for (int i = 0; i < len; i++) {
        // TODO: Приложете XOR операция върху text[i]
    }
}

// В main функцията добавете:
char data[] = "Secret";
char key = 'K';
xor_cipher(data, key);
printf("Encrypted: %s\n", data); // Внимание: Може да съдържа невидими символи!

```

```
xor_cipher(data, key);  
printf("Decrypted: %s\n", data);
```

ЗАДАЧА 3: Шифърът на Цезар (Модулна аритметика) - caesar.c

Цел: Да реализираме класически шифър, използвайки аритметика, и да се справим с превъртането на азбуката.

Условие:

Реализирайте функции за криптиране и декриптиране на главни латински букви ('A'-'Z').

- Формула за криптиране: $C = (P - 'A' + K) \% 26 + 'A'$
- Формула за декриптиране: $P = (C - 'A' - K + 26) \% 26 + 'A'$

Внимание: Не забравяйте + 26 при декриптирането, за да избегнете отрицателни резултати преди модула!

Скелет на кода:

```
C  
#include <ctype.h> // за isupper()  
  
char caesar_encrypt_char(char p, int k) {  
    if (isupper(p)) {  
        // TODO: 1. Превърнете p в индекс 0-25 (извадете 'A')  
        // TODO: 2. Добавете ключа k  
        // TODO: 3. Вземете модул 26  
        // TODO: 4. Върнете обратно към ASCII (добавете 'A')  
        return ( (p - 'A' + k) % 26 ) + 'A';  
    }  
    return p; // Ако не е буква (напр. цифра), връщаме без промяна  
}  
  
char caesar_decrypt_char(char c, int k) {  
    if (isupper(c)) {  
        // TODO: Реализирайте обратната логика.  
        // ВАЖНО: Използвайте (x - k + 26) % 26  
    }  
    return c;  
}
```

// В main() тествайте с текст "HELLO" и ключ 3.

Бонус: Модифицирайте функциите, така че да работят адекватно с малки и големи букви, както и цифри. Всички останали символи да се запазват в оригиналния си вид

ЗАДАЧА 4 (БОНУС): Работа с файлове

Цел: Да приложим знанията върху реални данни, а не само върху променливи в паметта.

Условие:

Напишете програма, която:

1. Чете съдържанието на файл `input.txt`.
2. Криптира го с XOR с ключ 'X'.
3. Записва резултата във файл `encrypted.bin`.

Помощен код за четене/писане:

```
C
FILE *in = fopen("input.txt", "rb");
FILE *out = fopen("encrypted.bin", "wb");
int ch;

while ((ch = fgetc(in)) != EOF) {
    // TODO: char encrypted = ch ^ key;
    // TODO: fputc(encrypted, out);
}

fclose(in);
fclose(out);
```

Въпроси за размисъл (Попълнете тук)

1. В Задача 2, ако съобщението беше "AAAAA", а ключът 'A', какъв щеше да е криптираният текст? Защо това е опасно?
 - *Отговор:*
2. Защо при Цезаровия шифър е безсмислено да имаме ключ 27?
 - *Отговор:*

Домашна работа

Задача 1: Caesar File Processor (CLI Tool)

Цел: Създаване на пълноценен инструмент за командния ред, който работи с файлови потоци.

Напишете програма `caesar_tool.c`, която приема аргументи от командния ред и извършва криптиране или декриптиране на файлове. Програмата трябва да работи по следния начин:

Входни файлове:

1. `plaintext.txt` – съдържа текста за обработка.
2. `key.txt` – съдържа само едно число (ключът, напр. 5).

Логика:

- Прочетете ключа от `key.txt` и го преобразувайте в `int` (полезни функции: `fscanf` или `fgets + atoi`).
- Прочетете входния файл символ по символ.
- Приложете Шифъра на Цезар (внимавайте с малки/големи букви и запазване на символи, които не са букви).
- Запишете резултата в `encrypted.txt` (при криптиране) или `decrypted.txt` (при декриптиране).

Бонус (Command Line Arguments): Направете така, че програмата да приема флагове от конзолата, вместо твърдо зададени имена на файлове. Примерно извикване:

```
./caesar_tool -e input.txt key.txt output.txt (за криптиране)
```

```
./caesar_tool -d output.txt key.txt restored.txt (за декриптиране)
```

Задача 2: Brute-Force Attack (Разбиване на шифър)

Цел: Да се усетите колко слаб е Цезаровият шифър.

Даден ви е следният криптиран текст (всичко е главни букви): "WKLV LV D VHFUHW
PHVVDJH"

Напишете програма `caesar_breaker.c`, която:

1. Приема този стринг.
2. Генерира и отпечатва всички 25 възможни отмествания (декриптира с ключ 1, после с 2, ..., до 25).
3. Оставете на човека (вас) да види кой ред има смисъл на английски език.

Бонус: Опитайте се да напишете интеграция с голям езиков модел, който има за задача да определи кой от текстовете е валиден

МИНИ-ПРОЕКТ: "Secure File Vault" (One-Time Pad)

Тема: Реализация на "съвършената" криптографска система (OTP) за произволни файлове (картинки, PDF, текст).

Описание: Шифърът на Цезар работи само с текст. XOR операцията обаче работи с **битове**, което означава, че може да криптира абсолютно всичко. Вашата задача е да напишете инструмент, който може да криптира и декриптира изображения.

Изисквания:

1. **Генератор на ключове (`keygen.c`):**
 - Приема дължина в байтове (напр. 1024).
 - Генерира файл `otp.key`, пълен със случайни байтове.
 - *Съвет:* Използвайте `/dev/urandom` (за Linux) или `rand()` (за Windows, само за учебни цели), за да напълните буфера.
2. **Криптираща програма (`otp_cipher.c`):**
 - Приема име на входен файл (напр. `image.png`) и файл с ключ (`otp.key`).
 - **ВАЖНО:** Проверява дали файлът с ключа е поне толкова голям, колкото входния файл. (Ако ключът е по-къс -> Грешка!).
 - Чете байт от файла и байт от ключа -> прави XOR -> записва в изходен файл (`image.enc`).

Тестов сценарий:

1. Вземете малка картинка (напр. `logo.png`).
2. Генерирайте ключ с размера на картинката.
3. Криптирайте я: `./otp_cipher logo.png otp.key logo.enc`.
4. Опитайте да отворите `logo.enc` – трябва да не се отваря или да е "шум".
5. Декриптирайте я (XOR отново): `./otp_cipher logo.enc otp.key logo_restored.png`.
6. Проверете дали `logo_restored.png` е идентична с оригинала.

Полезно:

- Функции като `fopen`, `fseek`, `ftell` (за размер на файл).
- Използвайте `return EXIT_SUCCESS;` и `return EXIT_FAILURE;` за професионален стил.

